

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

УДК 004.75

Отчет об исследовательском проекте на тему:
Системы балансировки нагрузки в гибридных распределенных системах

Выполнил студент:

группы #БПМИ236, 3 курса

Разухин Александр Сергеевич

Принял руководитель проекта:

Курочкин Илья Ильич
к.т.н., зав.лаб. Ц-1
ИППИ РАН

Москва 2026

Содержание

Аннотация	4
1 Введение	5
1.1 Описание предметной области	5
1.2 Актуальность и значимость	6
1.3 Цели и задачи	6
1.4 Состав результатов проекта	6
2 Литературный обзор	8
2.1 Обзор предметной области	8
2.2 Обзор существующих методов	8
2.3 Особенности BOINC	9
2.4 Сравнение подходов	9
3 Моделирование системы	11
3.1 Выбор способа моделирования	11
3.2 Топология системы	11
4 Подходы к тестированию системы и метрики качества	13
4.1 Эвристики распределения задач	13
4.2 Метрики оценки производительности	13
4.3 Процесс тестирования	14
5 Разработка алгоритма балансировки	15
5.1 Планировщик BOINC по умолчанию	15
5.2 Требования к алгоритму	17
5.3 Разработка планировщика задач в гетерогенной системе	17
5.4 Подбор оптимальных параметров для алгоритма	20
5.5 Сопоставление подходов в контексте BOINC	21
5.6 Ограничения разработанного подхода	21
6 Результаты тестирования и сравнение	23
6.1 Стабильная среда	23
6.2 Нестабильная среда: недоступность $f_{\text{off}} = 10\%$	25
6.3 Нестабильная среда: недоступность $f_{\text{off}} = 20\%$	26

6.4	Нестабильная среда: недоступность $f_{\text{off}} \approx 32\%$	27
6.5	Нестабильная среда: недоступность $f_{\text{off}} = 50\%$	28
6.6	Нестабильная среда: недоступность $f_{\text{off}} \approx 68\%$	29
6.7	Результаты экспериментов	29
7	Заключение	31
	Список литературы	33

Аннотация

В работе исследуется выдача заданий в гетерогенной среде BOINC с помощью локального ранжирования задач при запросе клиента. Реализована эвристика *hybrid*, учитывающая оценку длительности задачи, условную мощность узла и статистику доступности BOINC. Создан тестовый контейнерный стенд из шести клиентов четырёх различных ресурсных профилей. В каждом сценарии выполнено сравнение штатного планировщика и шести экспериментальных политик на наборе синтетических заданий. *Hybrid* превосходит *baseline* (штатный планировщик задач BOINC) в метриках пропускной способности в нестабильной среде, а также повышает значения индекса Jain по числу завершённых задач во всех шести основных сравнениях. Установлен компромисс между равномерностью числа задач, временем обработки их фиксированного набора и нагрузкой на узлы; описаны условия применимости результатов.

Ключевые слова

Распределенные системы, балансировка нагрузки

1 Введение

1.1 Описание предметной области

Предметная область проекта охватывает распределенные системы с гетерогенной инфраструктурой с основными компонентами:

- BOINC (Berkeley Open Infrastructure for Network Computing) — платформа для добровольных вычислений, объединяющая ресурсы разнородных устройств (серверов, кластеров, персональных компьютеров) в единую сеть [4]
- Кластерные ресурсы — группа вычислительных узлов под общим управлением
- Гетерогенные ресурсы — устройства с различными характеристиками: вычислительной мощностью, объемом памяти, доступностью и другими
- Балансировщик нагрузки — компонент, отвечающий за распределение вычислительных задач между узлами с учетом их возможностей и текущих состояний

Под гибридной распределённой системой в данной работе понимается объединение кластерных вычислительных ресурсов и автономных пользовательских устройств в рамках одного BOINC-проекта. Эти группы устройств совместно обрабатывают задания, но различаются возможностями управления, ресурсными ограничениями и режимами доступности. В предполагаемой архитектуре кластерный сегмент представляет собой управляемые вычислительные ресурсы, тогда как участие пользовательских устройств зависит от их владельцев и локальных настроек BOINC.

Также предметная область имеет следующие особенности:

- Отсутствие централизованного контроля за узлами системы
- Разнородность ресурсов каждого из узлов
- Различные режимы работы узлов (не все устройства доступны на протяжении 100% времени)
- Различный объем задач, время выполнения которых может варьироваться от нескольких минут до нескольких дней
- Необходимость минимизации накладных расходов для балансировщика

1.2 Актуальность и значимость

Актуальность данного проекта обусловлена множеством существующих на данный момент проблем, таких как:

- 1 Современные вычислительные задачи такие, как анализ больших данных, биоинформатика, требуют огромного количества ресурсов, что способствует росту числа гетерогенных распределенных систем
- 2 BOINC учитывает производительность, доступность и совместимость узлов, однако локальная выдача задач в ответ на запрос клиента не обеспечивает заданного глобального распределения нагрузки
- 3 Необходимость развития динамических алгоритмов балансировки в противовес существующим решениям с их статическими правилами
- 4 Исследование компромисса между скоростью выполнения набора заданий и распределением работы между разнородными клиентами

1.3 Цели и задачи

Целью данного проекта является разработка системы выдачи заданий для распределенной системы на платформе BOINC. Для ее достижения необходимо было выполнить следующие задачи:

- 1 Обзор систем балансировки нагрузки для кластеров и распределенных систем
- 2 Выбор системы моделирования и адаптация модели под специфику гибридной распределенной системы
- 3 Реализация/модификация системы балансировки нагрузки гибридной распределенной системы
- 4 Тестирование системы балансировки на модели гибридной распределенной системы
- 5 Анализ полученных результатов

1.4 Состав результатов проекта

- 1 Подробное описание методики моделирования гибридной распределенной системы

- 2 Подробное описание системы балансировки нагрузки гибридной распределенной системы
- 3 Анализ результатов моделирования системы
- 4 Подготовка подробного отчета о проделанной работе и проведенном тестировании

2 Литературный обзор

2.1 Обзор предметной области

Балансировка нагрузки отвечает за распределение работы между несколькими вычислительными ресурсами. Необходимо отличать маршрутизацию сетевых запросов от планирования длительных вычислительных заданий: для второго критически важны оценка трудоёмкости, ограничения ресурсов, сроки получения результата и доступность клиента-исполнителя. Поэтому классические методы серверной и облачной балансировки [3, 5] требуют адаптации к системе BOINC.

Статические методы используют заранее заданные правила, например циклический порядок или случайный выбор. Динамические методы используют сведения о текущем состоянии ресурсов; отдельные варианты допускают перенос работы, но миграция не является обязательным свойством динамического планирования [1].

В BOINC разделены серверная выдача заданий, запрос работы клиентом и локальное планирование выполнения. Эти механизмы постоянно взаимодействуют между собой, поэтому результат серверной эвристики зависит также от частоты запросов и настроек клиента [2].

2.2 Обзор существующих методов

Рассмотрим методы балансировки нагрузки, применимые к распределённым вычислительным системам с акцентом на гибридные конфигурации (платформы типа BOINC + стабильные кластерные сегменты). Среди критериев для них можно выделить: простота мониторинга, устойчивость к churn (неконтролируемое подключение / отключение узлов), гетерогенность ресурсов, величина накладных расходов, масштабируемость. Всего можно выделить несколько групп подходов к балансировке нагрузки:

- Классические статические методы (Round Robin, Random Strategy) — просты в мониторинге, однако не учитывают размер и время выполнения задач
- Классические динамические методы (Least Connections) — используют число соединений как приближение нагрузки и не учитывают вычислительную трудоёмкость заданий
- Адаптивные методы — балансировка происходит на основе метрик, вычисляемых в реальном времени; требуют постоянных вычислений
- Предиктивные ML-подходы — методы с обучаемой оценкой нагрузки; их качество зависит от обучающих данных и соответствия условий применения [6]

- Существующие решения для BOINC (CluBORun) — интеграция кластерных ресурсов вычисления [7]

Большинство классических и оптимизационных алгоритмов работают при допущении, что узлы доступны и их характеристики относительно стабильны. В средах типа BOINC это допущение неприменимо: изменчивость системы и разнородность устройств приводят к частому перераспределению задач между узлами. Если в гомогенных системах перенос задач - относительно стандартный процесс, в гибридных системах такой подход может привести к серьёзному росту накладных расходов из-за неоднородности устройств, поэтому в подобных системах правильным подходом считается исключение или сведение к минимуму подобных операций. Также нужно отметить, что для гибридных систем важно сохранять баланс между количеством собираемых метрик и их полезностью для планировщика, что труднодостижимо при подходах с глобальным мониторингом узлов. [5]

2.3 Особенности BOINC

Платформа BOINC использует модель client-initiated: вычислительный узел сам запрашивает работу через RPC. Scheduler сканирует массив feeder, оценивает совместимость задач с хостом и ранжирует кандидатов по score. Назначенная задача не мигрирует на другой host до таймаута или повторной репликации. Поле *rsc_fpop_est* задаёт оценку сложности, которая может расходиться с реальным временем выполнения - это важный источник ошибок для любой adaptive-политики [4]. Решение CluBoRun [7] интегрирует кластерные ресурсы в BOINC-проект, но не решает глобальную балансировку по классам устройств с учетом их мощности.

2.4 Сравнение подходов

Поймем, насколько применимы указанные подходы к гибридным системам и выделим их недочеты:

Таблица 2.1: Сравнение групп подходов к балансировке нагрузки в гибридных системах

Группа методов	Тип	Требования мониторинга	Цель подхода	Минусы
Классические статические	Простые	Низкие	Простое планирование	Не учитывают размер задач
Классические динамические	Реактивные	Низкие/средние	Сократить число одновременных задач на узле	Число соединений не отражает трудоёмкость
Адаптивные методы	С обратной связью	Средние или высокие	Снижение задержки и выравнивание нагрузки	Расходы на сбор и состояния
Предиктивные подходы	ML Обучаемые	Высокие	Минимизация целевой функции ошибки	Требуют данных и обучения
CluBORun	Интеграция кластера / устройств	Низкие/средние	Использование idle ресурсов, учёт доступности	Иная задача: интеграция ресурсов

Сопоставление демонстрирует, какая информация необходима разным группам методов для принятия решения о назначении задания. Простые статические правила требуют небольшого объёма данных, но они не используют оценку объёма задания. Адаптивные и обучаемые методы позволяют учитывать дополнительные характеристики за счет удорожания процессов сбора данных и настройки. Интеграция кластерных ресурсов решает задачу подключения вычислительного сегмента, тогда как ранжирование заданий определяет использование уже подключённых ресурсов. Таблица обосновывает выбор локальной эвристики как исследуемого подхода, хоть и не является экспериментальным доказательством её превосходства над другими группами методов.

Данная работа направлена на устранение выявленных ограничений планировщиков задач за счёт более точного учёта динамической доступности узлов и повышения общей производительности системы при рациональном использовании вычислительных ресурсов.

3 Моделирование системы

3.1 Выбор способа моделирования

Для проверки политик балансировки выбран локальный стенд на базе Docker Compose и реального планировщика BOINC 8.3.0, а не discrete-event симулятор или GPU-кластер. Такой выбор обусловлен следующими соображениями:

- Сохраняется полный RPC-цикл BOINC (feeder, feasibility checks, replication, обновление БД)
- Результаты воспроизводимы через скрипты и конфигурационные файлы из артефактов каждого запуска
- Гетерогенность задаётся через Docker cgroup и BOINC preferences без покупки реальной группы устройств с реальным GPU
- Синтетическая нагрузка задаётся встроенным приложением для генерации задач с параметром `—cpu_time`, отвечающим за нагрузку на процессор

Основной эксперимент - CPU-only: отсутствие физических устройств с собственным GPU не позволяет построить корректную модель, а варианты с разворачиванием виртуальных машин лишь имитируют наличие GPU.

3.2 Топология системы

Тестовый стенд состоит из 6 хостов-устройств с различными характеристиками:

- 2 кластерных элемента (4 CPU, 4 GiB RAM)
- 1 устройство ПК (2 CPU, 2 GiB RAM)
- 1 маломощный узел (1 CPU, 1 GiB RAM)
- 2 элемента - мобильных устройств (0.5 CPU, 0.5 GiB RAM)

Каждое устройство представляет собой независимый вычислительный хост; с точки зрения технической реализации каждый хост - это отдельный Docker контейнер, который конфигурационно настроен под конкретное число CPU. Данный выбор хостов с их настройками был обусловлен следующими требованиями к тестовому стенду:

- Стенд обязан содержать разнородные с точки зрения мощности устройства для моделирования гибридной среды

- Стенд обязан содержать как высокопроизводительные элементы (кластеры), так и низкопроизводительные (мобильные устройства)
- Стенд обязан иметь несколько одинаковых устройств для проверки балансировки нагрузки между однородными хостами (распределение задач между одинаковыми с точки зрения мощности клиентами)

Таким образом и была сформирована модель системы, используемая при тестировании.

Гибридность характеризует сочетание способов организации ресурсов, а гетерогенность - различия в их вычислительных характеристиках. Наличие узлов с разным числом CPU само по себе описывает гетерогенность, но не полностью воспроизводит гибридную архитектуру. Практическая цель работы состоит в исследовании выдачи заданий ресурсам обоих типов через общий механизм BOINC с учётом размера задания, характеристик клиента и его доступности.

В тестовом стенде кластерный сегмент представлен двумя клиентами профиля cluster, а автономные устройства - профилями desktop, low-power и phone. Такое соответствие является модельным: контейнеры воспроизводят ограничения ресурсов, но не отдельный кластерный планировщик или действия владельцев устройств (для мобильных). Стенд моделирует ресурсную гетерогенность кластерных и пользовательских узлов.

Протокол churn Отказы моделируются через docker compose pause/unpause на не-кластерных клиентах (кластеры выступают высокодоступными узлами в данном исследовании) с рассинхронизацией старта (разница в начале циклов отказа конфигурируется через *CHURN_STAGGER_SECO*). Каждый хост с отказами проходит несколько циклов online → pause → unpause. Доля offline в одном цикле:

$$f_{\text{off}} = \frac{T_{\text{offline}}}{T_{\text{online}} + T_{\text{offline}}}.$$

Например, при 25/12 получаем $f_{\text{off}} \approx 32\%$, при 40/10 - 20%, при 12/25 - 68%. Все события (включение / отключение хоста) логируются для последующего вычисления метрики Load CV.

Допущения тестового стенда Все шесть клиентов используют одну физическую машину и упрощённую сеть. CPU-квоты Docker и настройки BOINC моделируют ограничения ресурсов, но не различные микроархитектуры процессоров. Ограничения $\text{max_wus_to_send} = 1$ и $\text{max_wus_in_progress} = 2$ ограничивают захват очереди задач более быстрыми хостами. Эти допущения не отменяют А/В-сравнение политик на одном стенде, однако не позволяют напрямую экстраполировать абсолютные значения метрик на production систему.

4 Подходы к тестированию системы и метрики качества

4.1 Эвристики распределения задач

В качестве контрольных эвристик в проекте реализованы:

- LPT (Longest Processing Time first) - приоритет длинным задачам; проверяется гипотеза о сокращении общего времени завершения набора
- SJF (Shortest Job First) - приоритет коротким задачам; исследуется влияние приоритета коротких задач на среднее время выполнения и продолжительность обработки всего набора; при этом крупные задачи могут простаивать в очереди дольше

Slurm и Torque работают в другой модели управления ресурсами и не включены в экспериментальное сравнение: их полноценная интеграция потребовала бы другого стенда, а также они неприменимы к гетерогенной структуре BOINC. На стенде сравниваются baseline, hybrid и локальные реализации эвристик random, LPT, SJF, Round Robin и Weighted Least Loaded.

4.2 Метрики оценки производительности

Выбор метрик обусловлен необходимостью учета как общих показателей производительности, так и особенностей гетерогенной вычислительной среды.

- Пропускная способность (Throughput) - число выполненных задач за единицу времени. Формула: $T = \frac{N_{\text{completed}}}{\Delta t}$, где Δt - время от начала выполнения задач до его окончания
- Загруженность узлов - для узла j : $u_j = \frac{\text{cpu_time}_j}{\text{online_seconds}_j \cdot \text{ncpus}_j}$. Сводная метрика - Load CV $= \sigma(u_j)/\bar{u}$ - коэффициент вариации CPU-нагрузки; может расходиться с Jain
- Общая продолжительность выполнения задач - $\text{Makespan} = \max_i t_{\text{finish}}^{(i)} - \min_i t_{\text{create}}^{(i)}$
- Равномерность числа завершённых задач (Jain) - $J = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}$, где x_i - число завершённых задач на хосте i ; в расчёт попадают только хосты с $x_i \geq 1$
- Среднее время выполнения задачи (Service time) - среднее время от отправки задания клиенту до получения результата сервером: $\bar{S} = \frac{1}{N} \sum_{i=1}^N (t_{\text{received}}^{(i)} - t_{\text{sent}}^{(i)})$, где N - число успешно завершённых заданий. p95 - 95-й перцентиль того же распределения

Такой набор метрик позволяет оценить и скорость системы (throughput, makespan, service time), и качество распределения (Jain, Load CV). Отдельно анализируется согласованность типов задач с классами устройств (таблицы short/medium/long по хостам).

Среднее время выполнения характеризует этап после выдачи задания и включает ожидание на клиенте, выполнение, возможные паузы и задержку возврата результата. Оно не является чистым временем вычислений и не учитывает ожидание задания в серверной очереди до отправки. Поэтому показатель рассматривается совместно с `makespan` и пропускной способностью.

4.3 Процесс тестирования

Система тестировалась в шести сценариях режима: одна стабильная среда и пять вариантов `churn` с разными $(T_{\text{online}}, T_{\text{offline}})$. Метрики автоматически вычисляются скриптом `analyze_metrics.py` и сохраняются в файл `analysis/metrics.json`. Параметры прогона - в файл `manifest.json`.

В качестве источника задач использовалось приложение `uppercase` с параметром `—cpu_time`. Нагрузка: 36 задач - 18 short (5 s), 12 medium (30 s), 6 long (120 s) - достаточный набор для ограниченного во времени эксперимента. Основная цель алгоритма - оптимизировать распределение задач по хостам, направив long/medium на более мощные узлы, а short - на менее мощные, найдя компромисс с остальными метриками.

Таблица 4.1: Сценарии тестирования

Churn	Online (s)	Offline (s)	f_{off}
stable	—	—	0
heavy	45	5	10%
heavy	40	10	20%
heavy	25	12	32%
heavy	30	30	50%
heavy	12	25	68%

Различные сценарии охватывают стабильную работу и пять режимов периодической недоступности пользовательских устройств. Они позволяют сопоставить политики при разных уровнях доступности. Доля недоступности относится к одному полному циклу: фактическая доля времени в нерабочем состоянии в течение эксперимента может отличаться из-за рассинхронизации и неполных циклов. Поскольку между сценариями меняются обе длительности, результаты характеризуют конкретные режимы `churn`, а не изолированное влияние одного процента недоступности, которое добавлено в таблицу и далее скорее для наглядности.

5 Разработка алгоритма балансировки

Прежде чем описывать разработанный алгоритм, необходимо зафиксировать, как именно работает штатный планировщик в BOINC, поскольку именно он выступает baseline в экспериментах и служит границей, преодолеть которую данная работа ставит своей целью, придя к компромиссу по производительности.

5.1 Планировщик BOINC по умолчанию

BOINC использует pull-модель распределения задач: вычислительный узел самостоятельно инициирует RPC-запрос к планировщику и сообщает, сколько работы он готов принять (в секундах CPU, в числе задач или иначе). Планировщик не выбирает хост для задачи - он лишь отвечает на запрос данного клиента в данный момент. Назначенная задача не мигрирует на другой узел по достижении таймаута, при ошибке или при создании новой реплики. Это принципиально отличает BOINC от централизованных кластерных планировщиков (Slurm, Torque), где планировщик видит всю очередь задач и все узлы одновременно, и именно эти недостатки должны быть устранены или сведены к минимуму в рамках текущего проекта.

Очередь задач, доступная планировщику, - это не вся база данных, а кэш в памяти, который заполняет отдельный процесс под названием *feeder*. Он читает из БД информацию о задачах, готовых к отправке, и помещает их в массив фиксированного размера (по умолчанию до 100 слотов, параметр *shmem_work_items* задаётся конфигурационно). Каждый элемент содержит метаданные задачи (оценку сложности *rsc_fpop_est*, требования к памяти и диску), приоритет реплики *res_priority* и счётчик отказов *infeasible_count*. Планировщик при каждом запросе клиента сканирует этот массив; порядок обхода начинается со случайного смещения, чтобы не застревать на одних и тех же задачах.

Для выбора задач текущему хосту в выбранной конфигурации вызывается функция *send_work_score()*, логика которой сводится к двум шагам:

- 1 Сканирование и ранжирование - для каждого слота в shared memory подбирается совместимая версия приложения под платформу хоста, для каждой допустимой пары (*workunit*, *host*) вычисляется *score*, после чего все кандидаты сортируются по *score* по убыванию
- 2 Отправка - планировщик последовательно пытается выдать задачи из отсортированного списка, пока не исчерпан лимит RPC *max_wus_to_send*, не превышены квоты *max_jobs_in_progress* и пока хост ещё запрашивает работу

Отдельный проход выполняется для каждого типа процессора (GPU-типы, затем CPU).

Перед выдачей проверяется допустимость задания для хоста:

- Достаточно ли оперативной памяти и дискового пространства?
- Приложение входит в список, выбранный пользователем (хостом) в *project preferences*?
- Хост успеет завершить задачу до глобального дедлайна по ней?

Последняя проверка - ключевая для гетерогенной среды. Оценка длительности выполнения задачи:

$$\hat{t}_{ij} = \frac{\text{rsc_fpops_est}_i}{\text{projected_flops}_j \cdot \text{available_frac}_j},$$

Где *available_frac* - доля времени, когда клиент статистически доступен для BOINC. К этой оценке добавляется *estimated_delay* - время, которое хост уже занят текущими задачами. Если $\hat{t}_{ij} + \text{estimated_delay} > \text{delay_bound}$, задача для данного хоста в *score* не попадает. Следствие: мощные и стабильные узлы забирают длинные задачи; слабые и малодоступные - только короткие или остаются без задач.

Если быстрые проверки пройдены, выполняется *slow_check()* с запросами в БД: нет ли уже результата этой задачи на данном хосте. Только после этого задача добавляется в ответ клиенту.

Оценка каждого хоста считается исходя из нескольких эвристик:

- *Reliability* - при требовании надёжности проверяется признак *reliable* соответствующей версии приложения на хосте
- *LTT* - для *high - priority workunits* в режиме *batchacceleration* задачи идут только на *LTT*-хосты
- *Locality* - бонус, если входные данные задачи уже есть на хосте
- *res_priority* - приоритет реплики из БД добавляется напрямую к оценке
- *App preferences* - учёт пожеланий пользователя

Из этих факторов и складывается итоговый *score*.

Что *baseline* оптимизирует и чего не делает. Штатный планировщик оптимизирует локальную выгоду для текущего RPC клиента. При этом он не учитывает глобальную загрузку хостов, не различает классы устройств - только доступность конкретного хоста и его характеристики. Таким образом, возникает проблема: мощные хосты забирают любые задачи, которые им предлагает планировщик, а более слабым узлам достаются не самые простые с точки зрения требуемой вычислительной мощности задачи, из-за чего те вынуждены простаивать.

5.2 Требования к алгоритму

Исходя из обзора литературы, ограничений штатного планировщика BOINC и целей данной работы, к разрабатываемому алгоритму были сформулированы следующие требования:

- 1 Минимальная инвазивность - сохранить весь контракт штатного scheduler: feeder, feasibility checks, replication, locking shared memory, обновление БД. Меняется только функция ранжирования допустимых кандидатов
- 2 *Size — aware dispatch* - ключевая идея: эвристическое сопоставление оценки размера задачи с ресурсным профилем узла
- 3 *Churn — awareness* - учёт динамической доступности узла, чтобы не отправлять тяжёлые задачи на малодоступные хосты
- 4 Отсутствие миграции - не вводить перенос задач между хостами, поскольку такая возможность потребовала бы отдельного механизма переноса состояния приложения
- 5 Компромисс с распределением нагрузки - новый алгоритм не должен значительно повысить нагрузку на узлы
- 6 Сравнимость - помимо целевой политики реализовать контрольные алгоритмы (random, LPT, SJF, Round Robin, Weighted Least Loaded) для сравнения качества распределения и накладных расходов

5.3 Разработка планировщика задач в гетерогенной системе

Основная реализация находится в *sched_custom_load_balancer.cpp*, переключение режима выполняется в *sched_send.cpp*. При включённом флаге вместо встроенной вызывается функция *send_work_custom()*, которая повторяет структуру оригинальной *send_work_score()*:

- Сканирование *ssp* — *wu_results*[] со случайным offset
- Подбор *app_version* и *feasibility* через те же методы *get_score()*, *wu_is_infeasible_fast()*, *slow_check()*
- Сортировка кандидатов по score и отправка top-*N*

Отличие - в методе *custom_score()*, который заменяет штатный *JOB::get_score()* для вычисления итоговой оценки хоста.

Поскольку Docker-клиенты часто сообщают одинаковый p_flops , для hybrid введён коэффициент $host_speed_factor = v_j$ по числу CPU узла. Спрогнозированное время в этом случае: $\hat{t}_{ij} = estimate_duration(i, j)/v_j$.

Основной идеей было взять за основу оценку baseline-алгоритма и дополнить ее членами, явно учитывающими размер задачи, доступность хоста и "соответствие" мощности хоста размеру задачи. Целевая политика - hybrid, остальные служат контрольными для изоляции отдельных гипотез.

Контрольные политики Все контрольные политики используют защищённый *baseline score* ($100 \times s^{BOINC}$), чтобы штатные факторы (*reliability, locality*) сохраняли влияние, а экспериментальный член мог их сдвинуть:

Таблица 5.1: Реализованные политики

Политика	Суть ранжирования
baseline	штатный <code>send_work_score()</code>
random	baseline + детерминированный псевдослучайный шум от <i>result_id</i> и <i>host_id</i>
lpt	baseline + $\log(1 + \hat{t}_{ij})$ - длинные задачи первыми
sjf	baseline - $\log(1 + \hat{t}_{ij})$ - короткие задачи первыми
round_robin	baseline + циклический score по <i>result_id mod 6</i> и <i>host_id</i>
weighted_least_loaded	baseline + $\log(1 + v_j \cdot a_j / (L_j + 1))$, где L_j - число задач in-progress на хосте
hybrid	рукописный score

Здесь v_j - *host_speed_factor* (число CPU у хоста), a_j - *cpu_available_frac*.

В таблице зафиксированы различия в правилах ранжирования заданий для разных политик. В частности, политики LPT и SJF задают противоположные предпочтения по оценке длительности задания, Random вводит псевдослучайный порядок, а Round Robin использует циклический компонент оценки. Hybrid объединяет в себе признаки самого задания и клиента. Данные реализации сохраняют ограничения выдачи BOINC и рассматриваются как локальные варианты эвристик для того, чтобы разнообразить сравнение baseline и hybrid политик.

Разработанная политика Для пары (workunit i , host j) вычисляется составной score. Сначала определяются вспомогательные величины:

$$\begin{aligned} a_j &= \text{clamp}(\text{cpu_available_frac}_j, 0.15, 1.0), \\ \tilde{t}_{ij} &= \hat{t}_{ij}/a_j, \\ \rho_{ij} &= \text{clamp}(\tilde{t}_{ij}/T_{\text{target}}, 0.01, 100), \end{aligned}$$

где $T_{\text{target}} = 30$ секунд - целевое время выполнения (параметр *custom_lb_target_runtime*, близко к длительности medium-задач). Далее:

- $\text{size_affinity}_{ij} = -|\log \rho_{ij}| \cdot a_j$ - штраф за несоответствие размера задачи целевому runtime
- $\text{deadline_press}_i = T_{\text{target}}/\text{delay_bound}_i$ - бонус срочным задачам
- $\text{runtime_penalty}_{ij} = \log(1 + \tilde{t}_{ij}/T_{\text{target}})$ - штраф за длинные задачи на текущем хосте
- $\text{host_fit}_{ij} = (\log(1 + v_j) - \log(1 + \tilde{t}_{ij})) \cdot a_j^2$ - бонус за соответствие мощности хоста длительности задачи

Дополнительный штраф $P_{ij} = \text{slow_host_long_penalty}$ задается явно (скейлинг к промежутку [0, 3]):

$$P_{ij} = \begin{cases} 1.5 \cdot \frac{\hat{t}_{ij}}{T_{\text{target}}}, & T_{\text{target}} > \hat{t}_{ij} > T_{\text{target}}/2, \\ 0.5 \cdot \left(\frac{\hat{t}_{ij}}{T_{\text{target}}} - 1 \right), & \hat{t}_{ij} > T_{\text{target}} \\ 0, & \text{иначе.} \end{cases}$$

Baseline score масштабируется с учётом доступности хоста, что ослабляет влияние *reliability / locality* на нестабильных хостах и усиливает на стабильных - важно для сценариев с отказами узлов:

$$s_{ij}^{\text{hybrid}} = (10 + 10 \cdot a_j) \cdot s_{ij}^{\text{baseline}},$$

Таким образом, это слагаемое оказывается в интервале от 0 до 2.

Итоговый score выглядит так:

$$\begin{aligned} \text{score}_{ij} &= s_{ij}^{\text{hybrid}} + w_{\text{size}} \cdot \text{size_affinity}_{ij} + w_{\text{dl}} \cdot \text{deadline_press}_i \\ &\quad - w_{\text{rt}} \cdot \text{runtime_penalty}_{ij} + w_{\text{size}} \cdot \text{host_fit}_{ij} - w_{\text{rt}} \cdot \text{slow_host_long_penalty}_{ij}, \end{aligned}$$

Веса задаются в config.xml: $w_{\text{size}} = 2$, $w_{\text{dl}} = 5$, $w_{\text{rt}} = 0.5$.

Интерпретация элементов score

- *size_affinity* предпочитает эффективные задачи, по времени выполнения близкие к T_{target} . При фиксированном ρ модуль штрафа уменьшается, но сама ρ также зависит от доступности.
- На текущем стенде v_j для конкретного хоста постоянно: слагаемое $\log(1 + v_j)$ внутри *host_fit* не меняет порядок задач. Переменная часть *host_fit* предпочитает меньшую длительность; соответствие крупных задач мощным узлам задаётся совокупностью членов, прежде всего *size_affinity* и P_{ij} .
- *runtime_penalty* и *slow_host_long_penalty* не дают накапливать длинные задачи на мало-мощные узлы
- *deadline_press* предотвращает долгое ожидание взятия в работу для задач с коротким таймаутом
- Множитель $(10 + 10 \cdot a_j)$ взамен 100 у контрольных политик - компромисс: baseline-факторы BOINC сохраняют значимость, но дополнительные слагаемые также могут играть важную роль при формировании финальной оценки хоста

5.4 Подбор оптимальных параметров для алгоритма

Веса алгоритма ($w_{\text{size}} = 2$, $w_{\text{dl}} = 5$, $w_{\text{rt}} = 0.5$, $T_{\text{target}} = 30$ s) подбирались итеративно по серии прогонов baseline vs hybrid в нестабильной среде с отказами до получения устойчивого преимущества hybrid по равномерности распределения задач при приемлемой потере в общей продолжительности работы и пропускной способности. Ключевые решения:

- Установка множителя baseline в значение $(10 + 10 \cdot a_j)$ - дополнительные слагаемые получают влияние на оценку, но уже посчитанный внутри BOINC score не обнуляется, поскольку цель проекта модифицировать текущую систему, а не написать с нуля. Увеличение коэффициента приводит к слишком высокой корреляции между обеими политиками; излишнее уменьшение - к минимизации вклада в итоговую сумму и как следствие потере смысла. Оптимальным по метрикам оказался коэффициент от 15 до 30 (при фиксированных остальных параметрах), после выявления которого была добавлена привязка к доступности - решение для более качественного учета churn
- $T_{\text{target}} = 30$ s - получилось сопоставимо с длительностью medium-задач. Увеличение или уменьшение приводило к нарушению равномерности распределения нагрузки и значительному снижению скорости

- $w_{\pi} = 0.5$ - баланс между штрафом тяжелых задач на слабых хостах и допустимым makespan
- Увеличение w_{size} или w_{π} даже на 30-50% повысило нагрузку на узлы и увеличило общее время работы на 5-15%, уменьшение же вернуло поведение к baseline (основную часть оценки составляет baseline-score)

Исходя из изложенного выше, были зафиксированы параметры для разработанного эвристического алгоритма, после чего тот стал готов к полноценному тестированию.

5.5 Сопоставление подходов в контексте BOINC

Таблица 5.2: Классические методы и их реализация на pull-модели BOINC

Метод	Способ анализа	Ограничение в BOINC
Round Robin	Глобальная очередь	Только local RPC
Least Loaded	Мониторинг всех узлов	Только состояние текущего хоста
LPT / SJF	Централизованный порядок	Ранжирование только внутри feeder
Random	Равновероятный выбор	Детерминированный хэшем
Hybrid	Составная эвристика	Решение для одного клиента

Здесь уточняются границы экспериментального сравнения. Отметим, что в рассматриваемой реализации решение принимается планировщиком непосредственно в момент запроса заданий конкретным клиентом, поэтому ранжирование ограничено кандидатами, находящимися в feeder. Данные результаты характеризуют локальные правила выдачи в BOINC и не могут перенесены непосредственно в реальную систему.

5.6 Ограничения разработанного подхода

- Локальность решений - планировщик не видит глобального распределения задач между однородными хостами: они имеют схожий baseline score, поэтому возможна неравномерность распределения похожих задач между такими хостами, что на самом деле не так и важно при сохранении схожей нагрузки на однородных узлах
- Зависимость от оценок - $\hat{t}_{ij}$ строится на rsc_fpops_est , которое может расходиться с реальным CPU-time
- Компромисс нагрузки на узлы vs makespan - усиление $host_fit$ и штрафов за большие задачи на слабых узлах улучшает равномерность и распределение по типам устройств, но теоретически может увеличить нагрузку на более мощные узлы

- Нет миграции при churn - если хост перестал быть доступен с задачей in-progress, та ждёт таймаута и репликации; разработанный алгоритм лишь снижает вероятность отправки тяжёлых задач на нестабильные хосты до их назначения

6 Результаты тестирования и сравнение

Проведены прогоны для 6 сценариев по всем политикам. Для каждого сценария было сделано 5 прогонов, после чего метрики были усреднены. Ниже - метрики по каждому сценарию и сводное сравнение. В таблицах используются следующие столбцы:

- Policy - текущая политика
- Throughput - пропускная способность (задач в секунду)
- Service time - среднее время от отправки задания клиенту до получения результата сервером (в секундах)
- p95 - 95-й перцентиль времени выполнения задачи (в секундах)
- Makespan - общее время выполнения набора задач (в секундах)
- Load CV - коэффициент вариации нагрузки на узлы
- Jain - равномерность распределения задач

Ниже эксперименты и короткие выводы по ним:

6.1 Стабильная среда

Таблица 6.1: Метрики

Policy	Throughput	Service time	p95	Makespan	Load CV	Jain
baseline	0.117	59.8	140.2	252	0.368	0.725
round_robin	0.116	53.8	141.5	266	0.449	0.812
weighted_least_loaded	0.109	54.9	152.5	274	0.711	0.761
sjf	0.109	56.5	154.2	272	0.453	0.818
random	0.111	68.9	154.0	269	0.350	0.628
lpt	0.095	67.7	146.2	324	0.451	0.688
hybrid	0.108	69.8	157.0	268	0.358	0.923

По приведённой таблице политика baseline имеет наибольшую пропускную способность, равную 0.117 задачи/с и минимальный makespan, равный 252 с. Round Robin имеет наименьшее среднее время обработки задания - 53.8 с. У Random минимальный Load CV, равный 0.350, но относительно низкий Jain, равный 0.628. Таким образом, лидеры по скорости, равномерности CPU-нагрузки и равномерности числа задач не совпадают.

У hybrid makespan составляет 268 с, а среднее время выполнения задачи - 69.8 с: относительно baseline это увеличение приблизительно на 6.3% и 16.7% соответственно. Пропускная способность ниже на 7.7%. Одновременно Jain равен 0.923, а Load CV - 0.358 против 0.368 у baseline. Можно сказать, что это соответствует компромиссу между более равномерным распределением и скоростью обработки.

Таблица 6.2: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	12	10	5	3	4	2	36
round_robin	10	10	5	4	3	4	36
weighted_least_loaded	9	12	5	4	3	3	36
sjf	9	10	7	3	3	4	36
random	14	10	6	2	2	2	36
lpt	11	12	5	2	4	2	36
hybrid	8	8	6	3	6	5	36

В таблице отражено распределение задач между клиентами в рамках каждой политики. При baseline два кластерных узла выполняют 22 задачи из 36, или 61.1%, а при hybrid - 16 задач, или 44.4%. В вариантах Random и LPT на кластеры приходится соответственно 24 и 23 задачи. У hybrid диапазон числа задач на одном узле составляет от 3 до 8, тогда как у baseline - от 2 до 12. Это объясняет рост Jain у hybrid, но не означает более равномерной вычислительной нагрузки: преимущество у hybrid составляет менее 5% относительно baseline, а значит точно утверждать о прямой связи между более оптимальным распределением задач и менее равномерным распределением нагрузки нельзя.

Таблица 6.3: Число завершённых задач по типам и хостам

	Host				Σ		Host				Σ
		short	medium	long				short	medium	long	
baseline	cluster-01	7	4	1	12	hybrid	cluster-01	3	4	1	8
	cluster-02	5	2	3	10		cluster-02	1	2	5	8
	desktop-01	2	2	1	5		desktop-01	2	4	0	6
	low-power-01	1	2	0	3		low-power-01	1	2	0	3
	phone-01	3	1	0	4		phone-01	6	0	0	6
	phone-02	0	1	1	2		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

Разбиение по типам задач раскрывает различие, которое не было заметно в таблице 6.2. В приведённом распределении для baseline кластерные узлы выполняют 12 из 18 коротких задач и 4 из 6 длинных; ещё две длинные задачи выполняются на desktop и phone. При hybrid все 6

длинных задач выполняются кластерными узлами, а оба мобильных клиента получают только короткие задания. Это соответствует предусмотренному эвристикой сопоставлению класса задания с ресурсным профилем клиента.

При этом равенство общего числа задач не обеспечивает полностью одинакового набора: у hybrid каждый кластер завершает по 8 задач, однако длинные задания распределяются между ними в отношении 1 к 5. Таблица подтверждает разделение работы по классам устройств, но не равномерность нагрузки между одинаковыми узлами и не сокращение общего времени выполнения набора.

6.2 Нестабильная среда: недоступность $f_{\text{off}} = 10\%$

Churn: online 45 s, offline 5 s.

Таблица 6.4: Метрики

Policy	Throughput	Service time	p95	Makespan	Load CV	Jain
baseline	0.122	66.1	142.0	250	0.679	0.806
round_robin	0.121	55.7	137.8	252	0.561	0.878
weighted_least_loaded	0.113	54.7	156.0	270	0.622	0.761
lpt	0.115	71.4	157.8	258	0.318	0.643
random	0.121	69.8	144.0	254	0.432	0.639
sjf	0.118	55.2	155.0	265	0.468	0.837
hybrid	0.126	69.7	139.5	226	0.697	0.923

По результатам эксперимента наименьший makespan у hybrid - 226 с против 250 с у baseline, что соответствует сокращению на 9.6%. Его пропускная способность также максимальна - 0.126 задачи/с против 0.122 у baseline. Однако среднее время обработки задач выше: 69.7 с против 66.1 с, а Load CV составляет 0.697 против 0.679. Таким образом, сокращение времени обработки всего набора заданий не сопровождалось улучшением всех характеристик выполнения и нагрузки.

Минимальное среднее время обработки задания имеет Weighted Least Loaded - 54.7 с, а минимальный Load CV - LPT, 0.318. hybrid достигает максимального Jain, равного 0.923, однако это значение не согласуется с последующей таблицей количеств задач. Как и в прошлом случае, таблица не позволяет выделить политику, превосходящую остальные по всем показателям. Несмотря на это, hybrid обходит baseline по большинству метрик, хоть и проигрывая в части критериев другим эвристикам.

Таблица 6.5: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	11	9	5	4	4	3	36
round_robin	9	9	7	4	4	3	36
weighted_least_loaded	8	9	7	4	4	4	36
sjf	9	10	5	4	4	4	36
lpt	12	13	5	2	2	2	36
hybrid	8	10	5	3	5	5	36
random	9	15	6	2	2	2	36

При политике baseline кластерные узлы выполняют 20 задач, или 55.6% набора, а при hybrid - 18 задач, или 50%. У LPT и Random на кластеры приходится соответственно 25 и 24 задачи.

Таблица 6.6: Число завершённых задач по типам и хостам

baseline	Host	short	medium	long	Σ	hybrid	Host	short	medium	long	Σ
	cluster-01	7	3	1	11		cluster-01	3	0	5	8
	cluster-02	4	2	3	9		cluster-02	3	6	1	10
	desktop-01	2	2	1	5		desktop-01	1	4	0	5
	low-power-01	3	1	0	4		low-power-01	1	2	0	3
	phone-01	2	1	1	4		phone-01	5	0	0	5
	phone-02	0	3	0	3		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

В представленном распределении для baseline кластерные узлы выполняют 11 из 18 коротких и 4 из 6 длинных задач. Ещё две длинные задачи приходятся на desktop и phone. У hybrid все длинные задачи выполняются кластерными узлами, а оба мобильных клиента получают по пять коротких задач. Данное распределение соответствует заложенному предпочтению направлять крупные задания мощным узлам, сохраняя более простую работу для устройств с меньшими ресурсами.

В то же время у hybrid длинные задачи распределены между кластерами как 5 и 1, а средние - как 0 и 6. А значит, близость суммарного количества задач (8 vs 10), не означает схожей нагрузки. Таблица не позволяет утверждать, что вычислительная нагрузка одинаковых узлов выровнена или что именно это распределение стало причиной различия makespan.

Далее будут приведены результаты еще нескольких экспериментов с разной доступностью узлов, по которым будет сделан общий вывод.

6.3 Нестабильная среда: недоступность $f_{\text{off}} = 20\%$

Churn: online 40 s, offline 10 s.

Таблица 6.7: Метрики

Policy	Throughput	Service time	p95	Makespan	Load CV	Jain
baseline	0.119	62.5	143.5	255	0.477	0.844
round_robin	0.102	62.2	166.8	285	0.591	0.850
weighted_least_loaded	0.111	58.0	161.5	276	0.519	0.857
sjf	0.103	58.6	164.0	294	0.608	0.857
random	0.114	76.1	169.0	262	0.450	0.610
lpt	0.113	75.6	161.8	264	0.320	0.610
hybrid	0.121	73.1	162.8	235	0.558	0.873

В данном эксперименте у hybrid снова минимальный makespan, равный 235 с, и максимальная пропускная способность, равная 0.121 задачи/с. У baseline эти показатели составляют 255 с и 0.119 задачи/с. Разница makespan соответствует сокращению на 7.8%. Вместе с тем среднее время обработки задачи у hybrid выше: 73.1 с против 62.5 с, а p95 составляет 162.8 с против 143.5 с. Эти различия иллюстрируют разницу между общим временем завершения работы и продолжительностью обработки отдельных заданий.

Наименьшее среднее время обработки снова у Weighted Least Loaded - 58.0 с, а минимальный Load CV - вновь у LPT, 0.320. У hybrid Jain равен 0.873 и является наибольшим среди остальных политик, но Load CV выше baseline: 0.558 против 0.477. Таким образом, в представленном сравнении более равномерное число завершённых задач не означает более равномерной CPU-нагрузки на узлы.

6.4 Нестабильная среда: недоступность $f_{\text{off}} \approx 32\%$

Churn: online 25 s, offline 12 s.

Таблица 6.8: Метрики

Policy	Throughput	Service time	p95	Makespan	Load CV	Jain
baseline	0.121	66.3	152.0	243	0.329	0.806
round_robin	0.112	58.1	160.0	277	0.465	0.857
weighted_least_loaded	0.103	61.8	161.0	295	0.459	0.893
sjf	0.110	62.8	157.2	290	0.587	0.851
random	0.085	72.2	164.0	363	0.399	0.610
lpt	0.122	75.7	160.2	237	0.344	0.617
hybrid	0.123	78.0	156.0	234	0.554	0.871

В этом эксперименте hybrid вновь имеет минимальный makespan, равный 234 с, и максимальную пропускную способность, равную 0.123 задачи/с. Близкие значения у LPT - 237 с и 0.122 задачи/с. Относительно baseline с makespan 243 с сокращение у hybrid составляет 3.7%. Разница между hybrid и LPT равна всего 3 с; без увеличения числа запусков и расчета разброса данных она не позволяет установить устойчивое преимущество определенной политики.

При этом среднее время обработки задачи у hybrid, равное 78.0 с, является наибольшим среди политик. Минимальное среднее время имеет Round Robin - 58.1 с, а наибольший Jain у Weighted Least Loaded - 0.893 против 0.871 у hybrid. Представленные значения доказывают, что оценка только по makespan недостаточна: уменьшение времени обработки всего набора заданий может сочетаться с худшими характеристиками выполнения отдельных задач и распределения нагрузки.

6.5 Нестабильная среда: недоступность $f_{\text{off}} = 50\%$

Churn: online 30 s, offline 30 s.

Таблица 6.9: Метрики

Policy	Throughput	Service time	p95	Makespan	Load CV	Jain
baseline	0.104	84.4	168.0	287	0.494	0.806
round_robin	0.097	68.3	168.2	320	0.344	0.931
weighted_least_loaded	0.085	79.4	195.5	370	0.590	0.878
sjf	0.102	71.3	143.0	304	0.279	0.871
random	0.109	89.7	165.5	289	0.474	0.639
lpt	0.106	88.5	178.0	294	0.377	0.587
hybrid	0.108	81.2	169.0	269	0.622	0.900

В этом эксперименте минимальный makespan у hybrid - 269 с против 287 с у baseline, что соответствует сокращению на 6.3%. Среднее время выполнения задания также ниже: 81.2 с против 84.4 с, или примерно на 3.8%. Однако p95 этой же метрики практически не различается - 169.0 с против 168.0 с, а Load CV выше: 0.622 против 0.494. Следовательно, представленные преимущества в среднем времени и makespan не означают улучшения обработки как таковой или равномерности CPU-нагрузки на узлы.

Максимальная пропускная способность оказалась у Random - 0.109 задачи/с, минимальное среднее время выполнения задачи и максимальный Jain - у Round Robin, 68.3 с и 0.931. Минимальный Load CV на этот раз у SJF - 0.279. Данное сравнение демонстрирует отсутствие единой

политики, лучшей по всем представленным критериям, можно говорить лишь о преимуществах hybrid в сравнении с baseline.

6.6 Нестабильная среда: недоступность $f_{\text{off}} \approx 68\%$

Churn: online 12 s, offline 25 s.

Таблица 6.10: Метрики

Policy	Throughput	Service time	p95	Makespan	Load CV	Jain
baseline	0.100	104.1	215.0	306	0.498	0.812
round_robin	0.100	87.3	188.0	305	0.456	0.885
weighted_least_loaded	0.099	79.1	182.0	317	0.485	0.931
sjf	0.104	80.3	165.2	302	0.517	0.893
random	0.088	96.8	173.8	360	0.626	0.735
lpt	0.115	100.4	178.2	260	0.346	0.610
hybrid	0.107	102.3	215.0	287	0.624	0.915

Минимальный makespan в данном эксперименте имеет LPT - 260 с, а также максимальную пропускную способность, равную 0.115 задачи/с, и минимальный Load CV, равный 0.346. При этом его Jain равен 0.610 (худший среди всех политик) и является наименьшим среди представленных политик. Эти результаты показывают, что грамотное распределение заданий и выравнивание CPU-нагрузки на узлы не являются эквивалентными целями.

Для hybrid makespan составляет 287 с против 306 с у baseline, то есть сокращение на 6.2%, однако он уступает LPT по этому показателю. Среднее время обработки задания у hybrid немного ниже baseline - 102.3 с против 104.1 с, а p95 этой метрики одинаков и равен 215 с. Load CV выше - 0.624 против 0.498, несмотря на более высокий Jain. Минимальное среднее время обработки и максимальный Jain имеет Weighted Least Loaded - 79.1 с и 0.931 соответственно. По представленным значениям можно сделать вывод, что повышение недоступности не приводит к универсальному преимуществу hybrid; выбор политики зависит от критерия, которому отдаётся приоритет: общее время работы, среднее время обработки задания, равномерность распределения нагрузки и честность распределения задач между узлами.

6.7 Результаты экспериментов

Исходя из проведенных экспериментов, у hybrid makespan ниже baseline в пяти сценариях с недоступностью и выше в стабильной среде. Само сокращение составляет от 3.7% до 9.6%, однако

среднее время обработки задания ниже baseline только при недоступности 50% и около 68%, а Load CV во всех пяти сценариях с отказами выше. Таким образом, представленные таблицы описывают компромисс между общей продолжительностью обработки всех заданий, выполнением отдельных задач и равномерностью нагрузки на узлы, а не улучшение всех характеристик балансировки.

Таким образом, разработанный алгоритм смог обойти baseline на текущем тестовом стенде по скорости работы, уступив однако в равномерности нагрузки на узлы в сценариях с нестабильной средой, предположительно из-за излишней консолидации задач на хостах, соответствующих их объему.

7 Заключение

В ходе работы разработан прототип системы выдачи заданий для модели гибридной распределённой системы на платформе BOINC. Под гибридной системой в данной работе понимается совместное использование кластерных ресурсов под общим управлением и автономных пользовательских устройств в рамках одного проекта. Различия ресурсных возможностей и режимов доступности участников создают необходимость учитывать как характеристики конкретного клиента, так и объем задачи.

Рассмотрены подходы к балансировке нагрузки и ограничения для их применения в pull-модели BOINC. Реализована эвристика hybrid, которая дополняет встроенную оценку задачи признаками ее длительности, условной мощности и доступности клиента. Изменения затрагивают ранжирование задач при запросе клиентов и сохраняют проверки системы BOINC. Для сравнения реализованы локальные варианты эвристик Random, LPT, SJF, Round Robin и Weighted Least Loaded. Создан контейнерный стенд из шести клиентов четырёх ресурсных профилей; рассмотрены стабильный режим и пять режимов периодической недоступности.

Полученные в ходе экспериментов результаты описывают компромисс между скоростью обработки очереди задач и характеристиками их распределения по узлам. В частности, makespan у hybrid ниже baseline в пяти сценариях с недоступностью на 3.7-9.6%, но выше в стабильной среде. При этом среднее время выполнения задачи ниже baseline только при недоступности 50% и около 68%; в остальных сценариях оно выше. Метрика равномерности нагрузки на узлы Load CV во всех пяти режимах с периодическими отказами также выше у hybrid. Таким образом, даже в рамках приведённых чисел сокращение времени работы не означает одновременного улучшения времени выполнения отдельных заданий и равномерности CPU-нагрузки.

В рассмотренных для hybrid распределениях длинные задания полностью приходятся на кластерные узлы, а мобильные клиенты обрабатывают только короткие. Такое распределение соответствует заложенному в эвристику предпочтению сопоставлять размер задания с ресурсным профилем клиента. Вместе с тем состав набора задач для одинаковых узлов может существенно различаться. Поэтому схожее число завершённых задач не свидетельствует об одинаковой вычислительной нагрузке на узлы.

Область применимости исследования в данный момент сильно ограничена моделью стенда. Контейнеры воспроизводят ресурсные ограничения, но не все особенности управления кластерным сегментом и автономными устройствами. Все шесть контейнеров размещены на одной физической машине, используется синтетический набор заданий, а локальное ранжирование не обеспечивает глобального оптимума. Отсутствие оценки разброса результатов пока что не позво-

ляет установить статистическую значимость различий между политиками.

Практический результат работы состоит в реализации расширяемого механизма ранжирования заданий и тестового стенда для сравнения различных политик в BOINC. Цель разработки и проверки прототипа на выбранной модели выполнена, а вопрос об устойчивом выигрыше в реальной гибридной системе требует дальнейшего исследования. Следующие шаги включают в себя расширение объема экспериментальных данных, перенос клиентов с контейнерной формы на реальные устройства, а также более точное моделирование гибридной системы.

Список литературы

- [1] Ali M. Alakeel. «A Guide to Dynamic Load Balancing in Distributed Computer Systems». В: *International Journal of Computer Science and Network Security* 1.2 (2010).
- [2] David P. Anderson. «BOINC: A Platform for Volunteer Computing». В: *Journal of Grid Computing* 18 (2020), с. 99—122. DOI: [10 . 1007 / s10723 - 019 - 09497 - 9](https://doi.org/10.1007/s10723-019-09497-9). URL: [https : / / boinc . berkeley.edu/boinc_a_platform_for_volunteer_computing.pdf](https://boinc.berkeley.edu/boinc_a_platform_for_volunteer_computing.pdf).
- [3] Tony Bourke. *Server Load Balancing*. O'Reilly & Associates, 2001.
- [4] I. I. Kurochkin и А. А. Dolgov. «Load balancing in grid systems from personal devices on the BOINC platform». В: *XIV All-russian multi-conference on management issues of the MCPU-2021*. 2021.
- [5] Mazedur Rahman, Samira Iqbal и Jerry Gao. «Load Balancer as a Service in Cloud Computing». В: *IEEE 8th International Symposium on Service Oriented System Engineering (SOSE)*. 2014.
- [6] Lun Wang. «Low-Latency, High-Throughput Load Balancing Algorithms». В: *Journal of Computer Technology and Applied Mathematics* 10.6 (2024), с. 153—160.
- [7] М. О. Манзюк, О. С. Заикин и М. А. Посыпкин. «CluBORun: программный комплекс для использования свободных ресурсов вычислительных кластеров в BOINC-расчетах». В: *Информационные технологии и вычислительные системы* 4 (2014), с. 3—11. URL: [https : / / www.isa.ru/jitcs/images/documents/2014-04/3_11.pdf](https://www.isa.ru/jitcs/images/documents/2014-04/3_11.pdf).